

```

1: #include <stdint.h>
2: #include "bootx/bootx.h"
3: #include "hal/hal.h"
4: #include "kernel/internal/core/device_poll_internal.h"
5: #include "kernel/internal/core/graphics_service_internal.h"
6: #include "kernel/internal/core/kernel_boot_internal.h"
7: #include "kernel/internal/core/kernel_init_internal.h"
8: #include "kernel/internal/core/kernel_panic_internal.h"
9: #include "kernel/internal/proc/process_internal_base.h"
10: #include "kernel/internal/proc/process_types_internal.h"
11: #include "kernel/internal/proc/process_elf_internal.h"
12: #include "kernel/internal/core/tty_internal.h"
13: #include "kernel/public/core/console.h"
14: #include "kernel/public/driver/driver.h"
15: #include "kernel/public/input/input_focus.h"
16: #include "kernel/public/mem/pmm.h"
17: #include "kernel/public/mem/address_space.h"
18: #include "kernel/public/proc/job_control.h"
19: #include "kernel/public/proc/process.h"
20: #include "kernel/public/proc/scheduler.h"
21: #include "kernel/public/proc/sched_policy.h"
22: #include "kernel/public/sys/syscall.h"
23: #include "kernel/public/core/kprint.h"
24: #include "kernel/public/mem/vmm.h"
25: #include "lib/string.h"
26:
27: static volatile uint32_t timer_ticks;
28: static uint16_t g_kernel_boot_trace_row;
29: extern uint64_t g_current_user_raw_entry;
30:
31: #define shell_tty (*tty_virtual(0))
32:
33: enum {
34:     IRQ_DISPATCH_CONTINUE = 0,
35:     IRQ_DISPATCH_RESUME_FAULT = 1,
36:     IRQ_DISPATCH_RESUME_KERNEL = 0xffffffffffff0ull,
37:     IRQ_VECTOR_BASE = 32u,
38:     IRQ_VECTOR_TIMER = 32u,
39:     IRQ_VECTOR_KEYBOARD = 33u,
40:     IRQ_VECTOR_SERIAL = 36u,
41:     IRQ_VECTOR_MOUSE = 44u,
42:     IRQ_VECTOR_LAST = 47u
43: };
44:
45: static void kernel_boot_trace(const char *text);
46:
47: static void kernel_halt_forever(void) {
48:     for (;;) {
49:         hal_display_service_pending();
50:         hal_cpu_halt();
51:     }
52: }
53:
54: static void kernel_panic_init_exit(void) {
55:     tty_putc(&shell_tty, '\n', 0x0f);
56:     tty_write_str(&shell_tty, "KERNEL PANIC: init exited\n", 0x0f);
57:     device_poll_serial_write("\nKERNEL PANIC: init exited\n");
58:     hal_cpu_cli();
59:     kernel_halt_forever();
60: }
61:
62: static void kernel_run_console_shell_forever(struct vfs *vfs) {
63:     if (vfs == 0) {
64:         kernel_panic_init_exit();
65:     }
66: }

```

```

67:     for (;;) {
68:         kernel_boot_trace("kernel: init exited; starting console shell");
69:         if (!process_exec(vfs, "/cmd/ush --tty /dev/tty", 0, PROCESS_EXEC_AUTO)) {
70:             kernel_boot_trace("kernel: console shell exec failed");
71:             break;
72:         }
73:         kernel_boot_trace("kernel: console shell exited");
74:     }
75:     kernel_panic_init_exit();
76: }
77:
78: static uint64_t kernel_handle_user_exception(uint32_t vector, const struct exception
_frame *frame) {
79:     struct process_session *session;
80:     struct user_page_mapping *mappings;
81:     const struct process *proc;
82:     uint64_t fault_addr = 0;
83:     int32_t exit_code = -11;
84:
85:     if (frame == 0 || (frame->cs & 0x3u) != 0x3u) {
86:         return IRQ_DISPATCH_CONTINUE;
87:     }
88:     if (vector != 0u && vector != 6u && vector != 8u && vector != 13u && vector != 1
4u) {
89:         return IRQ_DISPATCH_CONTINUE;
90:     }
91:
92:     session = process_current_session();
93:     mappings = process_current_mappings();
94:     if (current_cpu_user_state()->nested_kernel_stack_depth != 0) {
95:         uint32_t index = current_cpu_user_state()->nested_kernel_stack_depth - 1u;
96:
97:         if (current_cpu_user_state()->active_sessions[index] != 0) {
98:             session = current_cpu_user_state()->active_sessions[index];
99:             mappings = current_cpu_user_state()->active_mappings[index];
100:             process_bind_session(session, mappings);
101:             if (session->address_space.user_cr3 != 0) {
102:                 (void)vmm_switch_root_or_fail(session->address_space.user_cr3);
103:             }
104:         }
105:     }
106:     proc = process_current();
107:     if (session == 0 || proc == 0 || proc->image_kind == PROCESS_IMAGE_NONE) {
108:         return IRQ_DISPATCH_CONTINUE;
109:     }
110:
111:     if (vector == 0u) {
112:         exit_code = -8;
113:     } else if (vector == 6u) {
114:         exit_code = -4;
115:     }
116:
117:     if (vector == 14u) {
118:         __asm__ __volatile__ ("mov %%cr2, %0" : "=r" (fault_addr));
119:         if (session->address_space.user_cr3 != 0 &&
120:             vmm_resolve_cow_fault(session->address_space.user_cr3,
121:                                   fault_addr,
122:                                   frame->error_code)) {
123:             return IRQ_DISPATCH_RESUME_FAULT;
124:         }
125:         if (process_handle_demand_page_fault(session,
126:                                               mappings,
127:                                               fault_addr,
128:                                               frame->error_code)) {
129:             return IRQ_DISPATCH_RESUME_FAULT;
130:         }

```

```
131:         kprint("proc: fatal user exception pid=%u vec=%u rip=%lx err=%lx cr2=%lx nam
e=%s\n",
132:             proc->pid,
133:             vector,
134:             frame->rip,
135:             frame->error_code,
136:             fault_addr,
137:             proc->name != 0 ? proc->name : "(unnamed)");
138:     } else {
139:         kprint("proc: fatal user exception pid=%u vec=%u rip=%lx err=%lx name=%s\n",
140:             proc->pid,
141:             vector,
142:             frame->rip,
143:             frame->error_code,
144:             proc->name != 0 ? proc->name : "(unnamed)");
145:     }
146:     process_exit_current(session, exit_code);
147:     return IRQ_DISPATCH_RESUME_KERNEL;
148: }
149:
150: static void kernel_boot_trace(const char *text) {
151:     uint16_t row = g_kernel_boot_trace_row;
152:     uint16_t col = 0;
153:     uint16_t width = console_width();
154:     uint16_t rows = console_rows();
155:
156:     if (text == 0) {
157:         return;
158:     }
159:     if (kprint_is_ready()) {
160:         kprint("%s\n", text);
161:         g_kernel_boot_trace_row = tty_cursor_row(&shell_tty);
162:         return;
163:     }
164:     if (row >= rows) {
165:         return;
166:     }
167:     tty_clear_row(&shell_tty, row, 0x0f);
168:     while (*text != '\0' && col < width) {
169:         tty_put_at(&shell_tty, row, col, *text++, 0x0f);
170:         col++;
171:     }
172:     if (g_kernel_boot_trace_row + 1u < rows) {
173:         g_kernel_boot_trace_row++;
174:     }
175: }
176:
177: static int kernel_feed_keyboard_event(const struct keyboard_event *event, const stru
ct syscall_frame *frame) {
178:     struct tty *target_tty;
179:     int ctrl_c;
180:     int ctrl_z;
181:     int sigint;
182:     int sigtstp;
183:
184:     if (event == 0 || event->keycode == KEYBOARD_KEY_NONE) {
185:         return 0;
186:     }
187:     device_poll_push_keyboard_event(event, &timer_ticks);
188:     if (event->pressed && event->alt) {
189:         uint32_t tty_index = TTY_VIRTUAL_COUNT;
190:
191:         if (event->keycode == KEYBOARD_KEY_F1) {
192:             tty_index = 0u;
193:         } else if (event->keycode == KEYBOARD_KEY_F2) {
194:             tty_index = 1u;
```

```
195:         } else if (event->keycode == KEYBOARD_KEY_F3) {
196:             tty_index = 2u;
197:         } else if (event->shift && event->keycode == KEYBOARD_KEY_1) {
198:             tty_index = 0u;
199:         } else if (event->shift && event->keycode == KEYBOARD_KEY_2) {
200:             tty_index = 1u;
201:         } else if (event->shift && event->keycode == KEYBOARD_KEY_3) {
202:             tty_index = 2u;
203:         }
204:         if (tty_index < TTY_VIRTUAL_COUNT && tty_switch_active(tty_index)) {
205:             input_focus_clear();
206:             target_tty = tty_active();
207:             if (target_tty != 0) {
208:                 device_poll_set_mouse_selection_console(&target_tty->console);
209:                 kprint_set_tty(target_tty);
210:             }
211:             return 0;
212:         }
213:     }
214:     target_tty = tty_active();
215:     if (target_tty == 0) {
216:         target_tty = &shell_tty;
217:     }
218:     ctrl_c = event->pressed && event->ctrl && event->keycode == KEYBOARD_KEY_C;
219:     ctrl_z = event->pressed && event->ctrl && event->keycode == KEYBOARD_KEY_Z;
220:     sigint = ctrl_c && job_tty_sigint(target_tty);
221:     sigtstp = ctrl_z && job_tty_sigtstp(target_tty, frame);
222:     if (sigint) {
223:         input_focus_clear();
224:         tty_write_str(target_tty, "^C\n", 0x0f);
225:     } else if (sigtstp) {
226:         input_focus_clear();
227:         tty_write_str(target_tty, "^Z\n", 0x0f);
228:     } else if (input_focus_owner_pid() != 0u) {
229:         return 0;
230:     } else if (ctrl_c) {
231:         tty_feed_key_event(target_tty, event);
232:     } else if (!ctrl_z) {
233:         tty_feed_key_event(target_tty, event);
234:     }
235:     return sigint || sigtstp;
236: }
237:
238: uint64_t irq_dispatch(uint32_t vector, const struct syscall_frame *frame) {
239:     uint8_t irq_line;
240:
241:     if (vector == IRQ_VECTOR_TIMER) {
242:         struct keyboard_event usb_event;
243:         struct keyboard_event uart_event;
244:         int usb_signal = 0;
245:
246:         hal_timer_notify_tick();
247:         timer_ticks++;
248:         device_poll_poll_usb_mouse_events(&timer_ticks);
249:         while (device_poll_poll_usb_keyboard_event(&usb_event)) {
250:             if (kernel_feed_keyboard_event(&usb_event, frame)) {
251:                 usb_signal = 1;
252:             }
253:         }
254:         while (device_poll_poll_uart_keyboard_event(&uart_event)) {
255:             if (kernel_feed_keyboard_event(&uart_event, frame)) {
256:                 usb_signal = 1;
257:             }
258:         }
259:         sched_on_timer_tick(timer_ticks);
260:         hal_irq_ack(0);
```

```
261:     if (usb_signal && frame != 0 && (frame->cs & 0x3u) == 0x3u) {
262:         return IRQ_DISPATCH_RESUME_KERNEL;
263:     }
264:     return IRQ_DISPATCH_CONTINUE;
265: }
266:
267: if (vector == IRQ_VECTOR_KEYBOARD) {
268:     struct keyboard_event event;
269:     int signal = 0;
270:
271:     event = device_poll_read_ps2_keyboard_event();
272:     signal = kernel_feed_keyboard_event(&event, frame);
273:     hal_irq_ack(1);
274:     if (signal && frame != 0 && (frame->cs & 0x3u) == 0x3u) {
275:         return IRQ_DISPATCH_RESUME_KERNEL;
276:     }
277:     return IRQ_DISPATCH_CONTINUE;
278: }
279:
280: if (vector == IRQ_VECTOR_MOUSE) {
281:     device_poll_handle_mouse_irq(&timer_ticks);
282:     hal_irq_ack(12);
283:     return IRQ_DISPATCH_CONTINUE;
284: }
285:
286: if (vector == IRQ_VECTOR_SERIAL) {
287:     struct keyboard_event event;
288:     int signal = 0;
289:
290:     device_poll_handle_uart_irq();
291:     while (device_poll_poll_uart_keyboard_event(&event)) {
292:         if (kernel_feed_keyboard_event(&event, frame)) {
293:             signal = 1;
294:         }
295:     }
296:     hal_irq_ack(4);
297:     if (signal && frame != 0 && (frame->cs & 0x3u) == 0x3u) {
298:         return IRQ_DISPATCH_RESUME_KERNEL;
299:     }
300:     return IRQ_DISPATCH_CONTINUE;
301: }
302:
303: if (vector < IRQ_VECTOR_BASE || vector > IRQ_VECTOR_LAST) {
304:     return IRQ_DISPATCH_CONTINUE;
305: }
306:
307: irq_line = (uint8_t)(vector - IRQ_VECTOR_BASE);
308: device_poll_handle_network_irq(irq_line);
309: hal_irq_ack(irq_line);
310: return IRQ_DISPATCH_CONTINUE;
311: }
312:
313: uint64_t kernel_prepare_user_frame_return(const struct syscall_frame *frame) {
314:     return sched_prepare_user_frame_return(frame);
315: }
316:
317: uint64_t kernel_panic_dispatch_exception(uint32_t vector, const struct exception_frame *frame) {
318:     uint64_t rc = kernel_handle_user_exception(vector, frame);
319:
320:     if (rc == IRQ_DISPATCH_RESUME_KERNEL || rc == IRQ_DISPATCH_RESUME_FAULT) {
321:         return rc;
322:     }
323:     kernel_panic_handle_exception(&shell_tty, g_current_user_raw_entry, vector, frame);
324:     return IRQ_DISPATCH_CONTINUE;
325: }
326:
327: void kernel_main64(const struct bootx_boot_info *boot_info) {
328:     const struct bootx_memmap_entry *memmap;
329:     struct vfs *vfs;
330:     uint64_t kernel_phys_base;
331:     int init_started;
332:
333:     hal_cpu_cli();
334:     hal_cpu_enable_sse();
335:     string_runtime_init();
336:     if (boot_info != 0 && boot_info->hdr.magic == BOOTX_MAGIC) {
337:         hal_display_load_font(boot_info);
338:         hal_display_init(&boot_info->console);
339:         kernel_gfx_init(&boot_info->console);
340:     }
341:     tty_virtual_init_all(0, (uint16_t)(console_rows() - 1u), 0x0f);
342:     device_poll_set_mouse_selection_console(&shell_tty.console);
343:     kprint_init();
344:     kprint_set_tty(&shell_tty); /* Route boot logs through kprint as early as possible. */
345:
346:     g_kernel_boot_trace_row = 1;
347:     tty_set_cursor(&shell_tty, g_kernel_boot_trace_row, 0);
348:     kernel_boot_trace("kernel: entered");
349:     string_memory_benchmark();
350:     if (!kernel_boot_info_valid(boot_info)) {
351:         kernel_boot_trace("kernel: bad boot info");
352:         kernel_halt_forever();
353:     }
354:
355:     memmap = (const struct bootx_memmap_entry *) (uintptr_t)boot_info->memmap;
356:     kernel_log_boot_info(boot_info);
357:     kernel_log_memmap(memmap, boot_info->memmap_count);
358:     kernel_phys_base = kernel_detect_phys_base(boot_info);
359:     kernel_boot_trace("kernel: paging init");
360:     hal_paging_init(kernel_phys_base);
361:     kernel_log_paging_info();
362:     kernel_boot_trace("kernel: pmm init");
363:     pmm_init(memmap, boot_info->memmap_count, kernel_phys_base, boot_info->kernel_phys_size);
364:
365:     kernel_reserve_boot_modules(boot_info);
366:     if (boot_info->console.type == BOOTX_CONSOLE_FRAMEBUFFER) {
367:         uint64_t framebuffer_size =
368:             (uint64_t)boot_info->console.pitch * boot_info->console.height;
369:         int framebuffer_wc =
370:             hal_paging_set_write_combining(boot_info->console.framebuffer_addr,
371:                                             framebuffer_size);
372:         kprint("paging: framebuffer=%lx size=%lx write_combining=%u\n",
373:               boot_info->console.framebuffer_addr,
374:               framebuffer_size,
375:               (uint32_t)framebuffer_wc);
376:     }
377:     (void)hal_display_enable_backbuffer();
378:     kernel_log_pmm_info();
379:
380:     kernel_boot_trace("kernel: block devices");
381:     kernel_init_storage_devices(boot_info);
382:
383:     kernel_boot_trace("kernel: tty/process/vfs");
384:     kprint_set_boot_time(&timer_ticks); /* Enable timestamp logging (SOSP feature) */
385:
386:     vfs = kernel_init_core_services(&shell_tty, &timer_ticks);
387:     if (vfs == 0) {
388:         kernel_boot_trace("kernel: core services failed");
389:         kernel_halt_forever();
390:     }
391: }
```

```
388:     }
389:     if (kernel_apply_root_cmdline(vfs, boot_info) > 0) {
390:         kernel_boot_trace("kernel: root cmdline applied");
391:     }
392:     kernel_boot_trace("kernel: discover drivers");
393:     (void)driver_discover_root(vfs, "/DRIVERS");
394:     (void)driver_load_all(vfs);
395:     (void)driver_init_all();
396:     syscall_init(&shell_tty, &timer_ticks, vfs, boot_info, memmap, boot_info->memmap
_count);
397:     timer_ticks = 0;
398:
399:     kernel_boot_trace("kernel: interrupts");
400:     kernel_init_interrupts();
401:
402:     kernel_boot_trace("kernel: start init");
403:     init_started = kernel_try_run_init(vfs, &shell_tty, &g_kernel_boot_trace_row, bo
ot_info);
404:     if (init_started) {
405:         kernel_run_console_shell_forever(vfs);
406:     }
407:     kernel_boot_trace("kernel: init missing");
408:
409:     for (;;) {
410:         hal_display_service_pending();
411:         hal_cpu_halt();
412:     }
413: }
```